WNDRVR
SUPPORT NETWORK

VXWORKS 7 ARM ARCHITECTURE GUIDE, SR0650

Copyright Notice

Copyright © 2022 Wind River Systems, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written permission of Wind River Systems, Inc.

Wind River, Simics, Tornado, and VxWorks are registered trademarks of Wind River Systems, Inc. Helix, Pulsar, Rocket, Titanium Cloud, Titanium Control, Titanium Core, Titanium Edge, Titanium Edge SX, Titanium Server, and the Wind River logo are trademarks of Wind River Systems, Inc. Any third-party trademarks referenced are the property of their respective owners. For further information regarding Wind River trademarks, please see:

www.windriver.com/company/terms/trademark.html

This product may include software licensed to Wind River by third parties. Relevant notices (if any) are provided for your product on the Wind River download and installation portals:

https://delivers.windriver.com/

https://windshare.usa.windriver.com/

Wind River may refer to third-party documentation by listing publications or providing links to third-party websites for informational purposes. Wind River accepts no responsibility for the information provided in such third-party documentation.

Corporate Headquarters

Wind River
500 Wind River Way
Alameda, CA 94501-1153
U.S.A.
Toll free (U.S.A.): +1-800-545-WIND
Telephone: +1-510-748-4100
Facsimile: +1-510-749-2010

For additional contact information, see the Wind River website:

www.windriver.com

For information on how to contact Customer Support, see:

www.windriver.com/support

VxWorks 7 ARM Architecture Guide, SR0650

11 September 2020

1. OVERVIEW

1. Supported ARM Cores

VxWorks® for ARM provides the Workbench 4 development tools and the VxWorks operating system for the ARM family of architectures. ARM is a compact core that operates at a low power level.

Table 1. ARM Processor Support

CPU Type	Description	32-bit Support?	64-bit Support?	ARM arch version
ARMARCH7	Includes ARMv7-A, ARMv7-R and ARMv8-A AArch32 cores	Yes	No	ARMv7-A ARMv7-R ARMv8-A AArch32
ARMARCH7M	Includes ARMv7-M cores	Yes	No	ARMv7-M
ARMARCH8A	Includes ARMv8-A AArch64 cores	No	Yes	ARMv8-A AArch64
CORTEX_A5	Includes Cortex-A5 cores	Yes	No	ARMv7-A
CORTEX_A7	Includes Cortex-A7 cores	Yes	No	ARMv7-A
CORTEX_A8	Includes Cortex-A8 cores	Yes	No	ARMv7-A
CORTEX_A9	Includes Cortex-A9 cores	Yes	No	ARMv7-A
CORTEX_A15	Includes Cortex-A15	Yes	No	ARMv7-A
CORTEX_A35_AARCH32	Includes Cortex-A35 AArch32 cores	Yes	No	ARMv8-A AArch32
CORTEX_A53_AARCH32	Includes Cortex-A53 AArch32 cores	Yes	No	ARMv8-A AArch32
CORTEX_A57_AARCH32	Includes Cortex-A57 AArch32 cores	Yes	No	ARMv8-A AArch32
CORTEX_A72_AARCH32	Includes Cortex-A72 AArch32 cores	Yes	No	ARMv8-A AArch32
CORTEX_M4	Includes Cortex-M4 cores	Yes	No	ARMv7-M

CPU Type	Description	32-bit Support?	64-bit Support?	ARM arch version
CORTEX_R5	Includes Cortex-R5 cores	Yes	No	ARMv7-R
CORTEX_A35	Includes Cortex-A35 cores	No	Yes	ARMv8-A AArch64
CORTEX_A53	Includes Cortex-A53 cores	No	Yes	ARMv8-A AArch64
CORTEX_A57	Includes Cortex-A57 cores	No	Yes	ARMv8-A AArch64
CORTEX_A72	Includes Cortex-A72 cores	No	Yes	ARMv8-A AArch64

Note: This release of VxWorks for ARM only supports little-endian configurations for ARM Architecture, including Cortex-A series, Cortex-M series and Cortex-R series.

ARMv7 Cortex-A

The ARMv7 Cortex-A architecture are application profiles that provide high performance execution. This release of VxWorks only supports the 32-bit ARM instruction set for the ARMv7-A profile.

ARMv8 Cortex-A

The ARMv8 Cortex-A architectures are application profiles that provide high performance execution. ARMv8 Cortex-A processors add support for a 64-bit data path and the AArch64 execution state, while retaining full backward compatibility with ARMv7-A in the AArch32 execution state. LLVM is the only compiler supported for 64-bit ARM.

ARMv7 Cortex-M

The ARMv7 Cortex-M architecture is a microcontroller profile that differs significantly from other ARM architectures, emphasizing smaller code size and better determinism over execution speed and performance. Real-time processes (RTPs) are not supported for Cortex-M with this release.

ARMv7 Cortex-R

The ARMv7 Cortex-R architecture is a real-time profile that is optimized for real-time, deterministic applications. This release of VxWorks supports both ARM and Thumb2 instruction sets for the ARMv7-R profile. The Cortex-R profile shares most of the architecture code with Cortex-A profile. Since there is no MMU on ARMv7-R, the MMU and VM descriptions are not applicable to Cortex-R. Real-time processes (RTPs) are not supported for Cortex-R with this release.

2. ARM32 CORTEX-A/CORTEX-R

1. Interface Variations

Particular features and functions that are specific to ARM targets.

These unique characteristics

- are available only on ARM targets,
- use parameters specific to ARM targets, or
- have special restrictions or characteristics on ARM targets.

For more complete documentation on these functions, see the individual reference entries.

Optimized Libraries on page 3 Most VxWorks libraries are compiled from portable C source code, but there are some libraries that are compiled from assembly language for better performance.
Restrictions on cret( ) and tt( ) on page 3 The cret( ) and tt( ) functions make assumptions about the standard prolog for functions.
cacheLib on page 4 The cacheLock( ) and cacheUnlock( ) functions always return **ERROR**. Use of the cache and use of the memory management unit (MMU) are closely linked on ARM processors.
dbgLib on page 4ARM supports both software breakpoint and hardware breakpoint.
dbgArchLib on page 5 If you are using the target shell, additional architecture-specific functions are available.
intALib on page 5
intArchLib on page 6ARM processors generally have no on-chip interrupt controllers to handle the interrupts multiplexed on the IRQ pin. Control of interrupts is a BSP-specific matter.
vmLib on page 6caching and virtual memory are linked on ARM processors.
vxALib on page 7
vxMemProbeArchALib on page 7 The vxMemProbe( ) function, which probes an address for a bus error, is supported by trapping data aborts.

1.1. Optimized Libraries

Most VxWorks libraries are compiled from portable C source code, but there are some libraries that are compiled from assembly language for better performance.

The following libraries are optimized for ARM targets:

- **bLib**—buffer manipulation library (including the swab( ) function)
- **dllLib**—doubly-linked list manipulation library
- **ffsLib**—find first bit set library
- **vxAtomicLib**—atomic operations library

Parent topic: Interface Variations on page 3

1.2. Restrictions on cret( ) and tt( )

The cret( ) and tt( ) functions make assumptions about the standard prolog for functions.

If functions are written in assembly language, or in another language that generates a different prolog, the cret( ) and tt( ) functions may generate unexpected results.

The VxWorks kernel is built without a dedicated frame pointer. This is also the default build option for user application code. As such, cret( ) and tt( ) cannot provide backtrace information and the tt( ) function cannot be used for backtracing kernel code.

The tt( ) function does not report the parameters to C functions as it cannot determine these from the code generated by the compiler.

> ⚠ Warning: The kernel is compiled without backtrace structures. For this reason, tt( ) does not work within the kernel functions, and cret( ) can sometimes work incorrectly. Breakpoints and single-stepping work, even if the code is compiled without backtrace structures.

Parent topic: Interface Variations on page 3

1.3. cacheLib

The cacheLock( ) and cacheUnlock( ) functions always return **ERROR**. Use of the cache and use of the memory management unit (MMU) are closely linked on ARM processors.

Consequently, if **cacheLib** is used, **vmLib** is also required. In addition, **cacheLib** and **vmLib** calls must be coordinated. For more information, see Memory Management on page 16. The definition of the symbolic constant **_CACHE_ALIGN_SIZE** is not related to the defined CPU type (the latter now defines an architecture). Rather, it is related to the cache type of the specific CPU being used. Therefore, code (such as device drivers) for which it is necessary to know the cache line size should use the variable **cacheArchAlignSize** instead.

Parent topic: Interface Variations on page 3

1.4. dbgLib

ARM supports both software breakpoint and hardware breakpoint.

When you set a software breakpoint, VxWorks replaces an instruction with a known undefined instruction. VxWorks restores the original code when the breakpoint is removed; if memory is examined or disassembled, the original code is shown. The instruction cache is purged each time VxWorks changes an instruction to a software break instruction.

A hardware breakpoint uses the processor's debug registers to set the breakpoint. The supported number of hardware breakpoint is defined by the ARM implementation. For example, there are six breakpoints and four watchpoints for Cortex-A9. If you are using the target shell, you can set hardware breakpoints with the bh( ) function, which takes the following arguments:

breakpointAddress

Specify the address

breakpointType

Specify when to break. Use one of the following values:

BRK_INST

Break when an instruction at this address is executed. (0x00)

BRK_DATAR

Break when data is read from this address. (0x01)

BRK_DATAW

Break when data is written to this address. (0x02)

WNDRVR

BRK_DATARW

Break when data is read from or written to this address. (0x03)

breakpointTask

Specifies the task to set. To specify all tasks, use 0.

count

The number of times to hit the breakpoint before the process breaks.

quiet

Specify **FALSE** to print debug information when the process breaks. Specify **TRUE** to break without printing debug information.

For more information, see the reference entry for bh( ).

A maximum number of hardware breakpoints can be set on the target system. This is a hardware limit and cannot be changed.

In order to maintain compatibility with hardware-assisted debuggers, the hardware breakpoint can be disabled using the _WRS_CONFIG_ARM_DBG_HARDWARE_BP VSB configuration option.

Parent topic: Interface Variations on page 3

1.5. dbgArchLib

If you are using the target shell, additional architecture-specific functions are available.

psrShow( )

Displays the symbolic meaning of a specified processor status register (PSR) value on the standard output.

cpsr( )

Returns the contents of the current processor status register (CPSR) of the specified task.

Parent topic: Interface Variations on page 3

1.6. intALib

intLock( ) and intUnlock( )

The intLock( ) function returns the I bit from the CPSR as the lock-out key for the interrupt level prior to the call to intLock( ). The intUnlock( ) function takes this value as a parameter. For ARM processors, these functions control the CPU interrupt mask directly. They do not manipulate the interrupt levels in the interrupt controller chip.

The VxWorks SMP kernel does not allow interrupt locking on all the CPUs in the system, and so these functions are not available in VxWorks SMP.

intCpuLock( ) and intCpuUnlock( )

These functions are called by a VxWorks task or interrupt service routine, and control whether interrupts are enabled or disabled on that context's local CPU.

The intCpuLock( ) function returns the I bit from the CPSR on the local CPU as the lock-out key for the interrupt level prior to the call to intCpuLock( ). The intCpuUnlock( ) function takes this value as a parameter. For ARM processors, these functions control the local CPU interrupt mask directly. They do not manipulate the interrupt levels in the interrupt controller chip.

WNDRVR

The VxWorks SMP allows interrupt locking on individual local CPUs in the system, and so these functions are available in VxWorks SMP. On the uniprocessor VxWorks, these functions are equivalent to intLock( ) and intUnlock( ).

> Note: On VxWorks SMP, the interrupts on the other CPUs in the system are not affected by these functions. Consequently, they are not suitable as a mutual exclusion mechanism unless all the tasks and/or interrupt service routines participating in the mutual exclusion have a single CPU affinity to the same CPU.

intIFLock( ) and intIFUnLock( )

The intIFLock( ) function returns the I and F bits from the CPSR as the lock-out key in an analogous fashion, and the intIFUnlock( ) function takes that value as a parameter. Like intLock( ) and intUnlock( ), these functions control the CPU interrupt mask directly. The intIFLock( ) function is not a replacement for intLock( ); it should only be used by code (such as FIQ setup code) that requires that both the IRQ and the FIQ be disabled.

Parent topic: Interface Variations on page 3

1.7. intArchLib

ARM processors generally have no on-chip interrupt controllers to handle the interrupts multiplexed on the IRQ pin. Control of interrupts is a BSP-specific matter.

All of these functions are connected by function pointers to functions that must be provided in ARM BSPs by a standard interrupt controller driver. For general information on interrupt controller drivers, see Wind River AppNote46, Standard Interrupt Controller Devices (available on the Wind River support website). For special requirements or limitations, see the appropriate interrupt controller device driver documents.

intEnable( ) and intDisable( )

The intEnable( ) and intDisable( ) functions affect the masking of interrupts in the BSP interrupt controller and do not affect the CPU interrupt mask.

intLockLevelSet( ) and intLockLevelGet( )

The intLockLevelSet( ) and intLockLevelGet( ) functions are not supported for ARM. The functions are present in this release but are not functional.

intVecBaseSet( ) and intVecBaseGet( )

The intVecBaseSet( ) and intVecBaseGet( ) functions are not supported for ARM. The functions are present in this release but are not functional.

intVecSet( ) and intVecGet( )

The intVecSet( ) and intVecGet( ) functions are not supported for ARM and are not present in this release.

intVecShow( )

The intVecShow( ) function is not supported for ARM and is not present in this release.

Parent topic: Interface Variations on page 3

1.8. vmLib

caching and virtual memory are linked on ARM processors.

Use of **vmLib** requires that **cacheLib** be included as well, and that calls to the two libraries be coordinated. For more information, see Memory Management on page 16.

Parent topic: Interface Variations on page 3

1.9. vxALib

mmuReadId( )

The mmuReadId( ) function is provided to return the processor ID on processors with MMUs that provide such an ID.

> Note: Do not call this function on CPUs that do not have this type of MMU. Doing so causes an undefined instruction exception.

vxTas( )

The test-and-set primitive vxTas( ) provides a C-callable interface to the ARM LDREX/STREX (load/store exclusive) instructions.

vxCpsrGet( )

The vxCpsrGet( ) function provides a C-callable interface to read the current CPSR.

Parent topic: Interface Variations on page 3

1.10. vxMemProbeArchALib

The vxMemProbe( ) function, which probes an address for a bus error, is supported by trapping data aborts.

If your BSP hardware does not generate data aborts when illegal addresses are accessed, vxMemProbe( ) does not return the expected results. The BSP can provide an alternative function by inserting the address of the alternative function in the global variable *_func_vxMemProbeHook* .

Parent topic: Interface Variations on page 3

2. Architecture Considerations

There are characteristics of ARM processors that you should keep in mind as you write a VxWorks application.

For comprehensive documentation on the ARM architectures and specific processors, see the ARM Architecture Reference Manual and the data sheets for your target processor.

Processor Mode on page 8 VxWorks for ARM executes mainly in 32-bit supervisor mode (SVC32).
Byte Order on page 8ARM CPUs include support for the little-endian byte order only. In this release of VxWorks for ARM, big-endian is not supported.
Hardware Breakpoints on page 8 Cortex-A7, Cortex-A9, and Cortex-A15 support hardware breakpoints.
ARM and Thumb State on page 8 VxWorks for ARM supports the 32-bit instruction set (ARM state) for all CPUs. Thumb2 is supported for Cortex-R. Thumb and ThumbEE are not currently supported.
Unaligned Accesses on page 8ARM Architecture Version 7 CPUs support unaligned data access. VxWorks doesn't enable strict alignment access.
Exceptions and Interrupts on page 9 When an interrupt or exception occurs on an ARM processor, the CPU switches to one of several exception modes, each of which has a number of dedicated registers.
Non-Zero-Based Exception Vectors on page 9BSPs with exception vectors that are not based at address zero can specify the location of the exception vectors by calling the excVecBaseSet( ) function from usrInit( ) with an address argument.
Exception Pointer Table Addressing on page 10BSPs that do not want to have the exception pointer table offset as 0x100 from the base address of the exception vectors can specify the offset of the exception pointer table by calling the excPtrTableOffsetSet( ) function from sysInit( ) with an offset argument.

Divide-by-Zero Handling on page 10 There is no native divide-by-zero exception on ARM architectures.
Floating-Point Support on page 10 VxWorks kernels and downloadable kernel modules (DKMs) for the CPU type ARMARCH7 can be compiled with hardware floating-point support enabled or disabled.
Vector Floating-Point Support on page 11 This release includes support for ARM vector floating-point coprocessors.
Implicit Vectorization and FPU-based Compiler Optimizations on page 12
Caches on page 13
Memory Management on page 16 In VxWorks 7, the BSP does not handle device memory space. Instead, this is handled in the device driver.
Physical Addressing Support on page 19 VxWorks is a 32-bit operating system; therefore, it supports 32-bit virtual addressing. However, some target hardware includes physical addressing space greater than 32 bits.
Expose the Target Memory Map Programmatically on page 37
Memory Layout on page 20 The VxWorks memory layout (real or virtual, as appropriate) is the same for all ARM processors.

2.1. Processor Mode

VxWorks for ARM executes mainly in 32-bit supervisor mode (SVC32).

VxWorks for ARM executes mainly in 32-bit supervisor mode (SVC32). When exceptions occur that cause the CPU to enter other modes, the kernel generally switches to SVC32 mode for most of the processing. Tasks running within a real-time process (RTP) run in user mode.

This release does not include support for the 26-bit processor modes, which are obsolete.

Parent topic: Architecture Considerations on page 7

2.2. Byte Order

ARM CPUs include support for the little-endian byte order only. In this release of VxWorks for ARM, big-endian is not supported.

Parent topic: Architecture Considerations on page 7

2.3. Hardware Breakpoints

Cortex-A7, Cortex-A9, and Cortex-A15 support hardware breakpoints.

However, because the debug registers for Cortex-A8 and Cortex-R5 are memory mapped, Cortex-A8 and Cortex-R5 do not support hardware breakpoints.

Parent topic: Architecture Considerations on page 7

2.4. ARM and Thumb State

VxWorks for ARM supports the 32-bit instruction set (ARM state) for all CPUs. Thumb2 is supported for Cortex-R. Thumb and ThumbEE are not currently supported.

Parent topic: Architecture Considerations on page 7

2.5. Unaligned Accesses

ARM Architecture Version 7 CPUs support unaligned data access. VxWorks doesn't enable strict alignment access.

For more information, see the *ARM Architecture Reference Manual*.

Parent topic: Architecture Considerations on page 7

2.6. Exceptions and Interrupts

When an interrupt or exception occurs on an ARM processor, the CPU switches to one of several exception modes, each of which has a number of dedicated registers.

In order to make the handlers reentrant, the stub functions that VxWorks installs to trap interrupts, and exceptions switch from exception mode to SVC (supervisor) mode for further processing. The handler cannot be reentered while executing in an exception because reentry destroys the link register. When an exception or base-level interrupt handler is installed by a call to VxWorks, the address of the handler is stored for use by the stub when the mode switching is complete. The handler returns to the stub function to restore the processor state to what it was before the exception occurred. Exception handlers (excluding interrupt handlers) can modify the state to be restored by changing the contents of the register set that is passed to the handler.

ARM processors do not, in general, have on-chip interrupt controllers. All interrupts except FIQs are multiplexed on the IRQ pin (see the section *Fast Interrupt (FIQ)*), below. Therefore, you must provide functions within your BSP to enable and disable specific device interrupts, to install handlers for specific device interrupts, and to determine the cause of the interrupt and dispatch the correct handler when an interrupt occurs. You install these functions by setting function pointers.

A device driver then installs an interrupt handler by calling vxbIntConnect( ).

Exceptions other than interrupts are handled in a similar fashion: the exception stub switches to SVC mode and then calls any installed handler. Handlers are installed through calls to excVecSet( ), and the addresses of installed handlers can be read through calls to excVecGet( ).

Interrupt Stacks

VxWorks for ARM uses a separate interrupt stack in order to avoid having to make task interrupt stacks big enough to accommodate the needs of interrupt handlers. The ARM architecture has a dedicated stack pointer for its IRQ interrupt mode. However, because the low-level interrupt handling code must be reentrant, IRQ mode is only used on entry to, and exit from, the handler; an interrupt destroys the IRQ mode link register. The majority of interrupt handling code runs in SVC mode on a dedicated SVC-mode interrupt stack.

Fast Interrupt (FIQ)

Fast interrupt (FIQ) is not handled by VxWorks. BSPs can use FIQ as they want, but VxWorks code should not be called from FIQ handlers. If this functionality is required, the preferred mechanism is to downgrade the FIQ to an IRQ by software access to appropriately-designed hardware which generates an IRQ. The IRQ handler can then make such VxWorks calls as are normally allowed from interrupt context.

Parent topic: Architecture Considerations on page 7

2.7. Non-Zero-Based Exception Vectors

BSPs with exception vectors that are not based at address zero can specify the location of the exception vectors by calling the excVecBaseSet( ) function from usrInit( ) with an address argument.

The address supplied as an argument is the base address of the exception vectors. In VxWorks 7, BSPs use the parameter VEC_BASE_ADRS to configure this argument. Cortex-R only supports high vector(0xffff0000) and low vector(0).

Parent topic: Architecture Considerations on page 7

2.8. Exception Pointer Table Addressing

BSPs that do not want to have the exception pointer table offset as 0x100 from the base address of the exception vectors can specify the offset of the exception pointer table by calling the excPtrTableOffsetSet( ) function from sysInit( ) with an offset argument.

This offset can be set for targets having restricted address space. If this function is not called, the exception pointer table remains based at the default offset of 0x100.

See Memory Layout on page 20 for more information.

Parent topic: Architecture Considerations on page 7

2.9. Divide-by-Zero Handling

There is no native divide-by-zero exception on ARM architectures.

In keeping with this, the Clang (LLVM) compiler toolchain does not synthesize a software interrupt for this event.

Parent topic: Architecture Considerations on page 7

2.10. Floating-Point Support

VxWorks kernels and downloadable kernel modules (DKMs) for the CPU type ARMARCH7 can be compiled with hardware floating-point support enabled or disabled.

See Vector Floating-Point Support on page 11 for details. RTPs for CPU type ARMARCH7 can be compiled with either hardware or software floating point support. The floating-point library used by VxWorks for ARM is licensed from ARM Ltd. For more information on the floating-point library, see:

http://www.arm.com/

The VSB is configured by default to use hardware FP. This configuration is the best choice for most users. It uses the FP coprocessor to do floating-point computing, rather than performing software-simulated floating point calculation.

When VxWorks is configured to use the software FP, it uses the software FP calling convention (ABI). The software FP ABI uses the general purpose registers rather than the FP coprocessor to pass floating-point parameters to functions, and return the floating-point result to the caller.

The extra cost for the software ABI is the movement and conversion between general purpose registers and FP coprocessor registers.

The following command can change the VSB configuration to use software FP. (Replacing _WRS_CONFIG_FP_ARMARCH7_soft_32 by _WRS_CONFIG_FP_hard in the following call configures VxWorks to use hardware FP).

vxprj vsb config -s -add _WRS_CONFIG_FP_ARMARCH7_soft_32=y

Return Status

The floating-point math functions supplied with this release do not set **errno**. However, the return status can be obtained by calling ieee_status( ).

The ieee_status( ) prototype is as follows:

```
unsigned int  ieee_status (unsigned int mask, unsigned int flags);
```

For example:

```
d = pow( 0,0 );
status =  ieee_status(FE_IEEE_ALL_EXCEPT, 0);
printf( "pow( 0, 0 )=%g,  ieee_status=%#x\n", d, status );
```

The use of the ieee_status( ) function is defined in the appropriate ARM compilation tools documentation, available from the ARM Ltd. website.

The return values are defined in the following file:

installDir **/vxworks-7/pkgs_v2/os/arch/arm/kernel/armv7a/h/arch/arm/arm.h**

The MASK for all valid exceptions is **FE_IEEE_ALL_EXCEPT**.

Parent topic: Architecture Considerations on page 7

2.11. Vector Floating-Point Support

This release includes support for ARM vector floating-point coprocessors.

Support is as follows:

Coprocessor	Supported CPUs
VFPv3-D16, VFPv3-D32	Cortex-A8 Cortex-A9 Cortex-R5
VFPv4-D16	Cortex-A5 Cortex-A7
VFPv4-D32	Cortex A7 Cortex-A15

For more information about these coprocessors (and the NEON coprocessor), see the reference documentation for your processor family available from ARM Ltd.

In its default state, the VxWorks kernel does not fully support NEON coprocessors. If your processor has a NEON coprocessor, the VFPv3-D16 or VFPv4-D16 coprocessor support can be used (if your processor supports it). However, this support does not save or restore all available VFP registers. For processors with a NEON coprocessor, you must configure your VxWorks kernel with the VFPv3-D32 or VFPv4-D32 VSB option to fully support NEON (all 32 double-word registers) when floating-point support is enabled.

VxWorks only supports the Advanced SIMD coprocessor when it is packaged with a VFPv3-D32 or VFPv4-D32 coprocessor (for example, a NEON coprocessor).

Enable Vector Floating-point

Vector floating-point support is enabled as follows:

- Define INCLUDE_HW_FP in your BSP or VxWorks project.
- If you are using Cortex-A8, Cortex-A9, Cortex-A9 MPCore, or Cortex-R5 define the VFPVARIANT build macro in your makefile or VxWorks project to be VFPv3.

- Add VX_FP_TASK to the options field of taskInit( ) or taskSpawn( ).
- If you need VFPv3-D32, VFPv4-D32, or NEON support, select the VSB configuration option as described in the section *Enable ARM VFPv3-D32/VFPv4-D32 Support*, below.

Currently, VFP exceptions are not detected or handled.

Enable ARM VFPv3-D32/VFPv4-D32 Support

A VSB option is available to support VFPv3-D32 and VFPv4-D32 (Advanced SIMD/NEON) floating-point coprocessors for ARM-based processors.

After enabling vector floating point, you must select the configuration option shown below.

The following example shows the VSB configuration option for VFPv3-D32, VFPv4-D32, and NEON support:

```
*
* arm Architecture Options
*
Enable VFPv3-D32/VFPv4-D32/NEON register context switch support?
(ENABLE_VFPV3_D32) [N/y/?]
```

Parent topic: Architecture Considerations on page 7

2.12. Implicit Vectorization and FPU-based Compiler Optimizations

The Clang (LLVM) compiler is capable of optimizing the code it produces, performing certain data-intensive operations using vector and/or floating point instructions, even when the application code is not written explicitly to imply this intent.

By default, VxWorks uses the -mno-implicit-float compiler option to suppress the generation of these code optimizations.

However, you can reenable the generation of these code optimizations by adding the option -mimplicit-float. When you do this, you must take care to spawn any VxWorks tasks that will by optimized by the compiler to use floating point or SIMD registers with the appropriate task option (VX_FP_TASK).

On 32-bit ARMv7A and ARMv7M architectures, an exception is raised if an execution context inadvertently executes a floating point instruction or performs a SIMD register access operation. If it is possible that this exception could occur in a task context, you must ensure the VX_FP_TASK option is specified when spawning that particular task. If, however, it is possible the exception could occur in an ISR context, then that particular ISR code must explicitly invoke the VxWorks API call fppSave( ) before the ISR floating point operations can occur. The ISR code must then explicitly invoke fppRestore( ) at the end of the ISR, after the floating point operations have completed their execution.

For detailed information about the compiler and library changes, see the *VxWorks 7 C11, C++14, C++17, and Compiler Migration Reference*.

Note:

In some scenarios, the VxWorks Clang/LLVM compiler will generate float instructions during compilation. If your application task's math operations do not appear to generate consistently correct results, it is likely that the processor floating point registers are being used by your application task and then overwritten again unexpectedly by other tasks when the kernel reschedules them. To resolve this situation, adjust the taskSpawn( ) call for the VxWorks task that is generating incorrect math results and add **VX_FP_TASK** as a task option. This will ensure that, if the floating point registers are being used by your application task, they will be protected when the kernel reschedules tasks. Your application task should then generate consistently correct math results.

Parent topic: Architecture Considerations on page 7

2.13. Caches

ARM processor cores have a variety of cache configurations. This section discusses these configurations and their relation to the ARM memory management facilities. The following subsections augment the information in the *VxWorks 7 Platform Programmer's Guide: VxWorks 7 Memory Model*.

ARM-based CPUs have one of three cache types: no cache, unified instruction and data caches, or separate instruction and data caches. Caches are also available in a variety of sizes. An in-depth discussion regarding ARM caches is beyond the scope of this document. For more detailed information, see the ARM Ltd. website at:

http://www.arm.com/

In addition to the collection of caches, ARM cores can also have one of three types of memory management schemes: no memory management, a memory protection unit (MPU), or a full page-table-based MMU. Detailed information regarding these memory management schemes can also be found on the ARM website. The following summarizes supported ARM cache and MMU/MPU configurations. Cache size is detected automatically during VxWorks initialization.

Core	Cache Size	Memory Management
Cortex-A5	4 KB to 64 KB (Level 1)	Page-table-based MMU
Cortex-A7 MPCore	8 KB to 64 KB (level 1) 0-1 MB (Optional On-chip Level 2)	Page-table-based MMU
Cortex-A8	16 KB to 32 KB (Level 1) 128 KB to 1 MB (On-chip Level 2)	Page-table-based MMU
Cortex-A9	16 KB to 64 KB	Page-table-based MMU
Cortex-A9 MPCore	16 KB to 64 KB	Page-table-based MMU
Cortex-A15 MPCore	32 KB (Level 1) 512 KB to 4 MB (On-chip Level 2)	Page-table-based MMU
Cortex-R5	4 KB to 64 KB (Level 1)	Region-based MMU

Note: For MPCore processors, the configuration specified in the table applies to each of the up to four CPUs available for the core. For more information on processor support, see Supported ARM Cores on page 1.

For all ARM caches, the cache capabilities must be used with the MMU (or MPU) to resolve cache coherency problems. When the MMU is enabled, the page descriptor for each page selects the cache mode, which can be cacheable or non-cacheable. You can configure this page descriptor by changing the platform function *xxxPhysMemInfoGet( )*. For example in platform fsl_imx, the function is *fslImx6PhysMemInfoGet( )* and the path is:

installDir **/vxworks-7/pkgs_v2/os/psl/freescale/fsl_imx/src/fslImx6.c**

For more information on cache coherency, see the cacheLib reference entry. For information on MMU support in VxWorks, see the *VxWorks 7 Platform Programmer's Guide: VxWorks 7 Memory Model*.

For MMU information specific to the ARM family, see 2.3.14 Memory Management, p. 17.

Not all ARM caches support cache locking and unlocking. Therefore, VxWorks for ARM does not support locking and unlocking of ARM caches. The cacheLock( ) and cacheUnlock( ) functions have no effect on ARM targets and always return ERROR.

The effects of the cacheClear( ) and cacheInvalidate( ) functions depend on the CPU type and on which cache is specified.

Implement External L2 Cache Support on page 14 The ARM Cortex-A5 and Cortex-A9 boards usually contain an external L2 cache (PL310).
ARM Cortex-A8 Cache on page 14
ARM Cortex-A5 and Cortex-A9 Cache on page 15 Cortex-A5 and Cortex-A9 have separate instruction and data caches.
ARM Cortex-A9 MPCore Cache on page 15 The ARM Cortex-A9 MPCore processor core contains up to four Cortex-A9 CPUs, each having separate instruction and data caches.
ARM Cortex A7 and Cortex A15 MPCore Cache on page 15 The ARM Cortex-A7 and Cortex-A15 MPCore processor cores contains up to four CPUs, each having separate Level 1 (L1) instruction and data caches.

Parent topic: Architecture Considerations on page 7

Implement External L2 Cache Support

The ARM Cortex-A5 and Cortex-A9 boards usually contain an external L2 cache (PL310).

These external L2 cache is supported by installing a struct extCache pointer. A couple of function pointers should be supplied, such as enable, disable, flush, clear, invalidate and pipe flush. For an example, refer to the following file:

installDir **/vxworks-7/pkgs_v2/os/arch/arm/kernel/drv/src/vxbFdtPl310L2Cache.c**

Parent topic: Caches on page 13

ARM Cortex-A8 Cache

The ARM Cortex-A8 has separate Level 1 (L1) instruction and data caches. Both are enabled by default. Both the data cache and the instruction cache can be set to copyback or write-through mode on a per-page basis.

The ARM Cortex-A8 may also contain a unified Level 2 (L2) cache. If the L2 cache is detected, VxWorks enables it by default and maintains coherency among the caches and memory. The L2 cache can operate in copyback or write-through mode on a per-page basis, and can be configured to follow the inner or outer cache policy. The L2 cache requires the L1 data cache to be enabled before it can operate. Also, when the L2 cache is operating, it may cache instructions even if the L1 instruction cache is disabled. See the *Cortex-A8 Technical Reference Manual* for details regarding L2 cache configuration and operation.

The following functions are available for controlling the Cortex-A8 on-chip L2 cache:

UINT32 cacheArchL2CacrGet (void)

This function returns the value of the Cortex-A8 L2 cache auxiliary control register, which contains information about L2 cache behavior.

void cacheCortexA8L2CacrSet (UINT32 cacr)

This function writes the given value to the Cortex-A8 L2 cache auxiliary control register, to configure L2 cache behavior. This function should be called before the L2 cache is enabled for the first time, such as in the BSP function sysInit( ). Note that this function must only be called with the processor in Secure state, which is the default state of the processor at reset. Also note that the TrustZone Security Extensions, and thus entering and exiting Secure state, are not supported.

void cacheCortexA8L2Enable (void)

This function enables the L2 cache. Note that L1 data cache must also be enabled before the L2 cache will actually be enabled. To enable both the L1 data cache and the L2 cache, use cacheEnable(DATA_CACHE). VxWorks 7.0 enable both L1 and L2 cache by default.

void cacheCortexA8L2Disable (void)

This function disables the L2 cache. To maintain coherency, this function also cleans and invalidates the L1 data cache and the L2 cache. The L1 data cache will remain in the same operational state (enabled/disabled). To disable both the L1 data cache and the L2 cache, use cacheDisable(DATA_CACHE). Note that the L2 cache is always disabled when the L1 data cache is disabled.

Parent topic: Caches on page 13

ARM Cortex-A5 and Cortex-A9 Cache

Cortex-A5 and Cortex-A9 have separate instruction and data caches.

Both caches are enabled and set to copyback by default. L1 cache for the Cortex-A5 and Cortex-A9 does not support write-through caching, but the L2 cache does. Cortex-A5 and Cortex-A9 processors can be connected with an external L2 cache, which is supported by VxWorks.

For information about L1 and L2 cache for Cortex-A5 and Cortex-A9, see Cached Page Table Entries on page 18.

Parent topic: Caches on page 13

ARM Cortex-A9 MPCore Cache

The ARM Cortex-A9 MPCore processor core contains up to four Cortex-A9 CPUs, each having separate instruction and data caches.

Both caches are enabled by default. Both of the data caches can be set to copyback or write-through mode on a per-page basis. The instruction cache is a read cache that is not coherent with stores to memory.

> ⚠ Warning: Although the VxWorks SMP provides the option to set the cache mode to write-through for Cortex-A9 MPCore, ARM hardware does not support this option. Therefore, setting the cache mode to write-through may result in unpredictable system behavior.

Parent topic: Caches on page 13

ARM Cortex A7 and Cortex A15 MPCore Cache

The ARM Cortex-A7 and Cortex-A15 MPCore processor cores contains up to four CPUs, each having separate Level 1 (L1) instruction and data caches.

Both caches are enabled by default. Both of the data caches can be set to copyback or write-through mode on a per-page basis. The instruction cache is a read cache that is not coherent with stores to memory.

The ARM Cortex-A7 MPCore optionally contains a unified Level 2 (L2) cache. The ARM Cortex-A15 MPCore contains a unified Level 2 (L2) cache. The L2 cache, which is automatically enabled when the L1 data cache is enabled, maintains coherency among the caches and memory. The strictly-enforced inclusion property with L1 data caches means that a line that resides in any of the L1 data caches must also reside in the L2 cache. For details regarding L2 cache configuration and operation, see the Cortex-A15 MPCore Technical Reference Manual.

> ⚠ Warning: Although VxWorks SMP provides the option to set the cache mode to write-through for Cortex-A7 and Cortex-A15 MPCore, ARM hardware does not support this option. Therefore, setting the cache mode to write-through can result in unpredictable system behavior.

The following functions are available for controlling the Cortex-A7 and Cortex-A15 MPCore L2 memory system:

UINT32 cacheArchL2CacrGet (void)

This function returns the value of the L2 cache control register, which contains information about L2 cache behavior.

void cacheArchL2CacrSet (UINT32 cacr)

This function writes the given value to the L2 cache control register, to configure L2 cache behavior. This function must be called before the MMU is enabled on any CPU. VxWorks 7.0 have done this during arch library initialization.

UINT32 cacheArchL2EcrGet (void)

This function returns the value of the L2 extended cache control register, which contains information about L2 memory system behavior.

void cacheArchL2EcrSet (UINT32 ecr)

This function writes the given value to the L2 extended cache control register.

Parent topic: Caches on page 13

2.14. Memory Management

In VxWorks 7, the BSP does not handle device memory space. Instead, this is handled in the device driver.

The xxxPhysMemInfoGet( ) function only gets basic memory information for the operating system. For the implementation, see the following:

installDir **/vxworks-7/pkgs_v2/os/psl/freescale/fsl_imx/src/fslImx6.c**

The memory information is supplied in the device tree memory node. For more information, refer to the VxWorks 7 BSP and Driver Guide.

Memory management is generally performed on small pages that are 4 KB in size. The ARM concept of large pages is not used. Sections and supersections may be used by target processors that support them.

ARM Large Physical Address Extension (LPAE) on page 16 The ARM architecture version 7 introduced a significant extension to memory management, Large Physical Address Extension (LPAE).
Variable Page Sizes on page 17 By default, addresses are mapped in blocks of 4 KB, also called small pages.
Dynamic Page Size Optimization on page 17A dynamic method for mapping addresses as sections or supersections (if the MMU supports this functionality) is available through the VxWorks **vmBaseLib** function, vmPageOptimize( ).
Address Space Identifiers and RTP Limitation on page 18ARM Architecture Version 7 MMUs support 255 unique address space identifiers (ASIDs), which are used to improve context switch performance with real-time processes (RTPs) by reducing the need to flush and invalidate caches.
Cached Page Table Entries on page 18 The ARM MMU in ARMv7 cores can support caching page table entries to reduce virtual-to-physical address translation time and improve system performance.
Cache and Memory Management Interaction on page 19 The caching and memory management functions for ARM processors are both provided on-chip and are very closely interlinked. In general, caching functions on ARM require the MMU to be enabled.
BSP Considerations for Cache and MMU on page 19 When you build a BSP, you select the instruction set by choosing the architecture (that is, by defining CPU to be ARMARCHx); the cache and MMU are then handled automatically in ARM architecture code. You do not need to supply any defines to inform ARM architecture about the cache and MMU.

Parent topic: Architecture Considerations on page 7

ARM Large Physical Address Extension (LPAE)

The ARM architecture version 7 introduced a significant extension to memory management, Large Physical Address Extension (LPAE).

The LPAE is an optional extension to the ARMv7-A architecture profile. With this extension, the physical addresses extend to 40-bit. This section describes VxWorks support for this extension with ARM architecture version 7 CPUs. For more information about LPAE, see the *ARM Architecture Reference Manual* (ARM DDI 0406 B), available on the ARM Ltd. website.

The LPAE added a new translation table format, the long-descriptor format. This format supports the assignment of memory attributes to memory pages, in increments of 4 KB, across the complete input address range. (The VxWorks implementation of this format does not support the assignment of memory attributes to blocks of memory.)

Parent topic: Memory Management on page 16

Variable Page Sizes

By default, addresses are mapped in blocks of 4 KB, also called small pages.

The properties of these pages are stored in MMU translation tables. It is also possible to map addresses in 1 MB sized sections or 16 MB sized supersections (if the functionality is supported by your target processor).

Support for larger page sizes allows virtual memory management to be more efficient with fewer translation table walks or translation lookaside buffer (TLB) misses. Dynamic and static methods for mapping larger page sizes provide mechanisms for increasing system performance.

Parent topic: Memory Management on page 16

Dynamic Page Size Optimization

A dynamic method for mapping addresses as sections or supersections (if the MMU supports this functionality) is available through the VxWorks **vmBaseLib** function, vmPageOptimize( ).

The **vmBaseLib**, vmPageOptimize( ) function allows default sized 4 KB MMU pages to be coalesced into 1 MB sections or 16 MB supersections (if the MMU supports this functionality) for contiguous memory blocks having the same attributes.

De-optimization is performed automatically when necessary. For example, if the attributes are changed for part of a memory block that is mapped to a 1 MB MMU page, it is broken up into 4 KB pages and the new attributes are applied to the requested pages only.

Note: MMU page sizes other than 4 KB, 1 MB, and 16 MB are not supported for this architecture.

ARM architecture support for vmPageOptimize( ) has limitations that are not described in the reference entry for the vmPageOptimize( ) function:

- Once a page optimization operation (for example, vmPageOptimize( )) is performed, attempts to change page attributes during an interrupt are limited in order to prevent MMU table corruption. Calls to vmStateSet( ) during interrupt processing fail if the call causes page de-optimization, and return **S_mmuLib_ISR_CALL_BLOCKED**. Also, calls to vmStateSet( ) during interrupt processing while page optimization or de-optimization is in process always fail, and return **S_mmuLib_ISR_CALL_BLOCKED**.
- The Virtual Memory Management special purpose attribute **MMU_ATTR_NO_BLOCK** cannot be used, because there is no support for locking pages in the MMU subsystem.

Optimization of the entire kernel memory space can be done automatically at startup by adding the **INCLUDE_PAGE_SIZE_OPTIMIZATION** component to a VxWorks project. (The **INCLUDE_MMU_OPTIMIZE** component is also required, but is automatically included by the project component configuration tool.)

Some blocks of memory used by VxWorks, such as those allocated for MMU page tables and for error detection and reporting, may prevent optimization of certain areas. This is because those blocks of memory may have different attributes than the rest of memory in the same 1 MB-aligned address range.

Parent topic: Memory Management on page 16

WNDRVR

Address Space Identifiers and RTP Limitation

ARM Architecture Version 7 MMUs support 255 unique address space identifiers (ASIDs), which are used to improve context switch performance with real-time processes (RTPs) by reducing the need to flush and invalidate caches.

Since one ASID is reserved for kernel use, 254 RTPs are supported. If no ASIDs are available, rtpSpawn( ) returns **ERROR**.

Parent topic: Memory Management on page 16

Cached Page Table Entries

The ARM MMU in ARMv7 cores can support caching page table entries to reduce virtual-to-physical address translation time and improve system performance.

When cached page table support is enabled, the MMU retrieves page table entries (PTEs) from the L1 or L2 cache (depending on memory page attributes, MMU configuration, and MMU hardware support). VxWorks enables cached page tables, by default, for ARMv7 cores in the ARM VxWorks kernel.

VxWorks provides a VSB build option for supported architectures to disable cached page table support in the VxWorks kernel if required.

The following example shows the VSB configuration option used to disable cached page table support:

```
*
* arm Architecture Options
*
Enable cached page tables optimization? (ENABLE_CACHED_PAGE_TBL) [Y/n/?]
(NEW)
```

To allow cached page table support to operate properly, page table memory is mapped with inner and outer cacheable, write-through attributes (when support for cached page tables is not available or enabled, page table memory is mapped as uncached), and the MMU is configured to use inner and outer cacheable, write-through memory for page tables. This memory configuration is selected by VxWorks because it is compatible with ARMv7 single core (such as Cortex-A8) and ARMv7 MPCore (such as Cortex-A9) MMUs and to avoid cache coherency issues when modifying page tables. For details about how the cached page table feature works, see the *ARM Architecture Reference Manual*.

The following table lists the cached page table entry support by architecture/core.

Architectures/Cores	Supported Modes
ARM Cortex-A5 (ARMv7)	Supported for ARM mode
ARM Cortex-A7 (ARMv7)	Supported for ARM mode
ARM Cortex-A8 (ARMv7)	Supported for ARM mode
ARM Cortex-A9 (ARMv7)	Supported for ARM mode
ARM Cortex-A15 (ARMv7)	Supported for ARM mode

Cortex-A5 and Cortex-A9 L1 cache does not support write-through caching. The Cortex-A5 and Cortex-A9 processors support an external L2 cache. To support cached page tables on the Cortex-A5 and Cortex-A9, the page table memory is configured as outer

write-through cacheable, inner non-cacheable. An L2 cache is required to support this configuration. If an L2 cache is not available, support for cached page tables should be disabled in a VSB build.

The L1 and L2 caches for Cortex-A7 and Cortex-A15 do not support write-through caching. To support cached page tables, the page table memory is configured as inner and outer write-back cacheable.

To specify that cached page tables are not supported by a BSP (required for BSPs based on architectures VxWorks does not support) a **common.vxconfig** file must be created in the BSP directory with the following contents:

```
# Indicate the BSP does not support cached PTEs
config UNSUPPORTS_CACHED_PAGE_TBL
bool

default y
```

Note: If you add the **common.vxconfig** file to your BSP, you will not be prompted to enable cached page table support when you configure your VSB.

Some features of the MMU that are required for cached page tables to function properly are implementation defined by the *ARM Architecture Reference Manual*, as specified by ARM. Review the TTBR register description and the L1 and L2 cache capabilities for the ARMv7 virtual memory system architecture (VMSA) implementation of your device and verify that cached page tables are supported for inner and outer cacheable, write-through memory.

Parent topic: Memory Management on page 16

Cache and Memory Management Interaction

The caching and memory management functions for ARM processors are both provided on-chip and are very closely interlinked. In general, caching functions on ARM require the MMU to be enabled.

Consequently, if cache support is configured into VxWorks, MMU support is also included by default. On some CPUs, the instruction cache can be enabled (in the hardware) without enabling the MMU; however this is not a recommended configuration.

Only certain combinations of MMU and cache-enabling are valid, and there are no hardware interlocks to enforce this. In particular, enabling the data cache without enabling the MMU can lead to undefined results. Consequently, if an attempt is made to enable the data cache by means of the cacheEnable( ) function before the MMU has been enabled, the data cache is not enabled immediately. Instead, flags are set internally so that if the MMU is enabled later, the data cache is enabled with it. Similarly, if the MMU is disabled, the data cache is also disabled until the MMU is reenabled.

Parent topic: Memory Management on page 16

BSP Considerations for Cache and MMU

When you build a BSP, you select the instruction set by choosing the architecture (that is, by defining CPU to be ARMARCHx); the cache and MMU are then handled automatically in ARM architecture code. You do not need to supply any defines to inform ARM architecture about the cache and MMU.

Parent topic: Memory Management on page 16

2.15. Physical Addressing Support

VxWorks is a 32-bit operating system; therefore, it supports 32-bit virtual addressing. However, some target hardware includes physical addressing space greater than 32 bits.

ARMv7 supports 32-bit virtual addressing and 64-bit physical addressing size (**PHYS_ADDR**). For Cortex-A7 and Cortex-A15 with LPAE enabled, it supports 40 **PHYS_ADDR** bits; otherwise, it supports 32 **PHYS_ADDR** bits.

Parent topic: Architecture Considerations on page 7

2.16. Expose the Target Memory Map Programmatically

The BSP statically defines the memory regions that are available to VxWorks while it is running on this target. For example, on the **ti_sitara_a8** BSP, the memory map for the **am335x-evmsk** board is defined in the file **am335x-evmsk.dts**.

```
memory
    {
    device_type = "memory";
    reg = <0x80000000 0x10000000>; /* on board 256MB */
    };
```

The memory map is also available programmatically to the VxWorks application, BSP and driver code. The VxWorks API function sysMemDescGet( ) must be called to retrieve the memory map configuration while the target is running.

For example:

```
    LOCAL PHYS_MEM_DESC sysPhysRamDesc[32];
    LOCAL STATUS success;

    success = sysMemDescGet(MEM_DESC_RAM, 0, sysPhysRamDesc);
```

Parent topic: Architecture Considerations on page 7

Parent topic: Architecture Considerations on page 26

2.17. Memory Layout

The VxWorks memory layout (real or virtual, as appropriate) is the same for all ARM processors.

The following figure shows memory layout, labeled as follows:

System Memory Pool

Size depends on size of the system image. The memPoolEndAdrs( ) function returns the end of the free memory pool.

System Image

VxWorks itself (three sections: text, data, and bss). The entry point for VxWorks is at the start of this region. This is configured within the BSP.

Initial Stack

Initial stack for usrInit( ), until usrRoot( ) is allocated a stack. This is configured within the BSP.

Early MMU Table

MMU Table setup before kernel initialized.

DTB Relocation

Copied DTB data.

Exception Message

ASCII string of fatal exception message.

Boot Line

ASCII string of boot parameters.

Exception Pointers

Pointers to exception functions, which are used by the vectors.

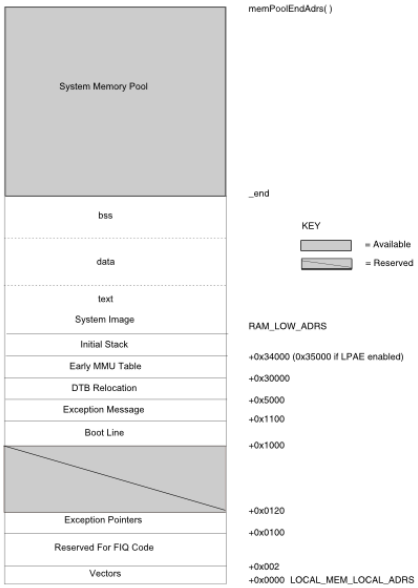
FIQ Code

Reserved for FIQ handling code.

Vectors

Table of exception/interrupt vectors.

The default location of his section can be specified by calling the excVecBaseSet( ) function. For more information, see Non-Zero-Based Exception Vectors on page 9. If this address is modified, the location of the exception pointers section is also altered. In this case, the new base location for exception pointers is also offset by the base address for the vectors. If the default exception pointer table offset is used, the new base location for exception pointers will be 0x100 plus the base address of the vectors section. The offset may also be provided using excPtrTableOffsetSet( ).



All addresses shown are relative to the start of memory for a particular target board. The start of memory (corresponding to 0x0 in the memory layout diagram) is defined as **LOCAL_MEM_LOCAL_ADRS** under **INCLUDE_MEMORY_CONFIG** for each target.

Parent topic: Architecture Considerations on page 7

3. Migrating Your BSP

In order to convert a VxWorks BSP from an earlier VxWorks 6.x release to VxWorks 7, you must make certain architecture-independent changes.

This includes making changes to custom BSPs designed to work with a VxWorks 6.x release and not supported or distributed by Wind River.

For additional migration information, see the *Migration Guide, Edition 5*.

4. Reference Material

Comprehensive information regarding ARM hardware behavior and programming is beyond the scope of this document. ARM Ltd. provides several hardware and programming manuals for the ARM processor on its website.

http://infocenter.arm.com/help/index.jsp

Wind River recommends that you consult the hardware documentation for your processor or processor family as necessary during BSP development.

ARM Development Reference Documents

The information given in this section is current at the time of writing; should you decide to use these documents, you may want to contact the manufacturer for the most current version.

- *ARM Architecture Reference Manual*, ARM DDI 0100 I.

  This document describes the architecture in general, including architectural standards for instruction bit fields. More specific information is found in the data sheets for individual processors, which conform to different architecture specification versions.
- *ARM Architecture Reference Manual, ARMv7-A and ARMv7-R edition*, ARM DDI 0406 B.
- *ARM System Architecture*, by Steve Furber. Addison-Wesley, 1996. ISBN 0-201-403352-8.
- *ARM Procedure Call Standard* (APCS), a version of which is available on the Internet. Contact ARM for information on the latest version.

3. ARM CORTEX-M

1. Unique Variations

1.1. Optimized Libraries

Most VxWorks libraries are compiled from portable C source code, but there are some libraries that are compiled from assembly language for better performance.

The following libraries are optimized for ARMv7-M targets:

bLib

Buffer manipulation library (including the swab( ) function).

dllLib

Doubly-linked list manipulation library.

ffsLib

fFnd first bit set library.

vxAtomicLib

Atomic operations library.

1.2. Restrictions on cret( ) and tt( )

VxWorks for ARMv7-M has the same restrictions on cret( ) and tt( ) as ARMv7 Thumb-2 state.

The cret( ) and tt( ) functions make assumptions about the standard prolog for functions. If functions are written in assembly language, or in another language that generates a different prolog, the cret( ) and tt( ) functions can generate unexpected results.

The VxWorks kernel is built without a dedicated frame pointer. This is also the default build option for user application code. Therefore, cret( ) and tt( ) cannot provide backtrace information, and the tt( ) function cannot be used for backtracing kernel code.

The tt( ) function does not report the parameters to C functions, as it cannot determine these from the code generated by the compiler.

Note: The kernel is compiled without backtrace structures. For this reason, tt( ) does not work within the kernel functions, and cret( ) can sometimes work incorrectly. Breakpoints and single-stepping work, even if the code is compiled without backtrace structures.

1.3. cacheLib

Cache libraries are supported by the architecture layer in VxWorks 7 for ARMv7-M.

The ARMv7-M architecture does not define an internal cache controller. However, some Cortex-M processors have an external cache controller. The cache library (**cacheLib**) calls the external cache driver directly for the ARMv7-M architecture.

1.4. dbgLib

VxWorks for ARMv7-M supports both software breakpoints and hardware breakpoints.

WNDRVR

When you set a software breakpoint, VxWorks replaces an instruction with a known undefined instruction. VxWorks restores the original code when the breakpoint is removed; if memory is examined or disassembled, the original code is shown.

A hardware breakpoint uses the processor's debug registers (flash patch and breakpoint unit, FPB) to set the breakpoint. If you are using the target shell, you can use the bh( ) function to set hardware breakpoints. The bh( ) function only supports **BRK_INST** (instruction hardware breakpoint) in parameter type.

1.5. dbgArchLib

If you are using the target shell, additional architecture-specific functions are available.

psr( )

Returns the contents of the processor status register (PSR) of a specified task.

basepri( )

Returns the contents of the processor interrupt mask registers (BASEPRI) of a specified task.

psrShow( )

Displays the symbolic meaning of a specified processor status register (PSR) value on the standard output.

bh( )

Sets the hardware breakpoint.

1.6. intALib

intLock( ) and intUnlock( )

The intLock( ) function returns the value of the BASEPRI register as the lock-out key for the interrupt level prior to the call to intLock( ). The function intUnlock( ) takes this value as a parameter. These functions control the CPU interrupt mask directly. They do not manipulate the interrupt levels in the interrupt controller chip.

intIFLock( ) and intIFUnLock( )

These functions are unsupported on VxWorks for ARMv7-M.

1.7. intArchLib

The functions in **intArchLib** are connected by function pointers to functions that are initialized by the VxBus legacy interrupt library.

The real operation functions are provided in ARMv7-M BSPs by a nested vector interrupt controller (NVIC) driver, which is managed by a separate VxBus driver. If the VxBus legacy interrupt library (**isrLib**) has not been initialized (that is, _func_vxbIntConnect = NULL), these functions return **ERROR**.

The following is a short list of functions, for more information see **intArchLib** in *VxWorks 7 Kernel Core OS*.

intArchConnect( )

This function is a public API to connect the given C handler for the given external interrupt, if possible. This function is called by the public intConnect( ) API as a backup mechanism when the associated VxBus API (vxbIntConnect( )) is not used.

intArchDisconnect( )

This function is a public function to disconnect the given C handler from the given external interrupt, if possible. This function is called by the public intDisconnect( ) function as a backup mechanism when the associated VxBus function (vxbIntDisconnect( )) is not used.

intEnable( )

This function is a public API to enable the given interrupt, if possible, using VxBus legacy interrupt support (**_func_vxbIntEnable** function pointer).

intDisable( )

This function is a public API to disable the given interrupt, if possible, using VxBus legacy interrupt support (**_func_vxbIntDisable** function pointer).

intVecTableWriteProtect( )

This function is a public API that, although meant to write protect the vector table, is not supported and returns **ERROR** for ARMv7-M.

intRegsLock( )

This function is a public API to set the interrupt status as disabled in the given register set and return the previous interrupt lock state.

intRegsUnlock( )

This function is a public API to set/restore the interrupt status of the given register set to the interrupt lock state provided.

The following public APIs are not supported, and do nothing in ARMv7-M.

- intLockLevelSet( )
- intLockLevelGet( )
- intVecBaseSet( )
- intVecBaseGet( )
- intLevelSet( )

1.8. vmLib

The virtual memory library is not supported on VxWorks for ARMv7-M.

1.9. vxALib

vxALib functions have some ARM-specific characteristics.

mmuReadId( )

In VxWorks for ARMv7-M, this function returns the contents of the CPUID base register, which is valid whether there is a supported MPU or not.

vxTas( )

The test-and-set primitive, vxTas( ), provides a C-callable interface to the ARM Cortex-M **LDREX**/**STREX** (load/store exclusive) instructions.

1.10. vxMemProbeArchLib

The vxMemProbe( ) function, which probes an address for a bus error, is supported by trapping data aborts.

If your BSP hardware does not generate data aborts when illegal addresses are accessed, vxMemProbe( ) does not return the expected results. For ARMv7-M, a bus fault exception is generated when probing an address for a bus error.

Your BSP can provide an alternative function by inserting the address of the alternative function in the global variable *_func_vxMemProbeHook* .

WNDRVR

2. Architecture Considerations

This section describes characteristics of ARMv7-M. For comprehensive documentation on the ARM architecture and on specific processors, see the *ARM Architecture Reference Manual* and the data sheets for your target processor.

Note: For the features of Thumb-2 state, see the reference for Thumb-2.

Processor Mode on page 26 The Cortex-M processor supports two operation modes, thread mode and handler mode.
Byte Order on page 26ARM Cortex-M CPUs include support for both little-endian and big-endian byte orders. The libraries in this VxWorks release provide support for little-endian only.
ARMv7-M and Thumb State on page 26 VxWorks for ARMv7-M only supports the Thumb-2 instruction set.
Unaligned Accesses on page 27 Unaligned 32-bit accesses are allowed on ARMv7-M CPUs.
Exceptions and Interrupts on page 27 The ARMv7-M architecture implements a new exception model, which is different from the traditional ARMv7 architecture.
Non-Zero-Based Exception Vectors on page 28BSPs with exception vectors that are not based at address zero can specify the location of the exception vectors by calling the excVecBaseSet( ) function from usrInit( ) with an address argument.
Exception Pointer Table Addressing on page 28 Exception pointer table addressing is not supported in VxWorks for ARMv7-M.
Divide-by-Zero Handling on page 28 This exception is enabled (using the configuration and control register) during system initialization to generate a use fault.
Software Floating-Point Support on page 29 For the Cortex M3 core, hardware floating-point is unsupported.
Vector Floating-Point Support on page 29 For the Cortex-M4F core, vector floating-point is supported.
Memory Layout on page 29 The VxWorks memory layout for ARMv7-M (real or virtual, as appropriate) shares the same labels as that of the ARM memory layout.
Expose the Target Memory Map Programmatically on page 37

2.1. Processor Mode

The Cortex-M processor supports two operation modes, thread mode and handler mode.

When an exception or interrupt occurs, the processor enters handler mode automatically and always runs with privileged access. Thread mode is entered upon reset or upon returning from an exception or interrupt.

Parent topic: Architecture Considerations on page 26

2.2. Byte Order

ARM Cortex-M CPUs include support for both little-endian and big-endian byte orders. The libraries in this VxWorks release provide support for little-endian only.

Parent topic: Architecture Considerations on page 26

2.3. ARMv7-M and Thumb State

VxWorks for ARMv7-M only supports the Thumb-2 instruction set.

When using function pointers with the Thumb-2 instruction set, you must ensure that the least significant bit (LSB) of the address that is pointed to is set to one. There are several macros that provide this functionality. These macros are defined as no-ops for the ARM instruction set, and only update the specified value if built for Thumb-2.

_WRS_CHANGE_INSN_SET(p)

Sets the LSB of p. The macro assumes p is of type **FUNCPOINTER**, and does not return a value.

_WRS_CHANGE_INSN_SET_PTR(p)

Returns p with the LSB set to one and cast to type **FUNCPOINTER**.

_WRS_REV_INSN_SET_CHANGE(p)

Returns p with the LSB set to zero and cast to type **FUNCPOINTER**.

Parent topic: Architecture Considerations on page 26

2.4. Unaligned Accesses

Unaligned 32-bit accesses are allowed on ARMv7-M CPUs.

For this reason, unaligned access faults are not enabled in the configuration control register for the ARMv7-M processor in **cpuInitALib.s**.

Parent topic: Architecture Considerations on page 26

2.5. Exceptions and Interrupts

The ARMv7-M architecture implements a new exception model, which is different from the traditional ARMv7 architecture.

There are various types of exceptions in the ARMv7-M architecture, including:

- Reset
- NMI
- Hard Fault
- Memory Management Fault
- Bus Fault
- Usage Fault
- SVCall
- Debug Monitor
- PendSV
- SysTick

For more information on these exceptions, see the ARMv7-M Architecture Reference Manual.

Exception preemption is implemented based on the priority level in the ARMv7-M architecture. The priority level is split into preemption priorities (group priorities) and sub-priorities. The lower numbers are higher priority. The priority of exceptions is configurable, except for reset, NMI, and hard fault. The fixed priorities for the highest priority exceptions, in descending order are:

- -3 for reset
- -2 for NMI
- -1 for hard fault

During initialization of the exception handling subsystem (see excVecInit( )), the group priority of exceptions is set to the default priorities value defined in *dfltExcPriority[ ]*.

When an exception occurs on an ARMv7-M processor, the CPU switches to one of several exception types defined in the exception vector table. SysTick is treated as an interrupt in VxWorks for ARMv7-M. PendSV is treated as a specific exception used to implement a context switch. For other types of exceptions, except reset, a low-level/assembly stub (excStub) is installed by VxWorks to trap the exceptions. Upon taking an exception, the hardware automatically saves a basic context (R0-R3, R12, LR, PC, and PSR) into the stack being used at the time of the exception. It then switches to handler mode, and uses the main stack pointer. The default exception

handlers defined in *excHandlerTbl[ ]* are used by the stub. If the exception is recoverable, control is returned from the default exception handler to the exception stub. Otherwise, if the exception is not recoverable, the default exception handler reports fault information.

The following public APIs are available:

excVecBaseSet( )

This function specifies the location of the exception vector table. The default exception vector table location is typically at address 0. Note that although there is an alignment requirement on the address of the exception vector table, it is not validated by this function because it is dependent on the number of interrupts supported by the processor (that is, the maximum size of the table). Therefore, the alignment must be validated by the BSP. For ARMv7-M, the maximum vectors of the table can be obtained using the **CORTEXM_INTCTLR_GET_NUM_VECTORS** macro. The maximum size of the table is four times the maximum number of vectors.

excVecGet( )

This function returns the address of the C handler that is installed for a given exception.

excVecSet( )

This function installs the C handler for a given exception.

excVecPrioSet( )

This function sets the priority of the given system exception, excluding the PendSV and SysTick interrupts. If the priority can be changed (**MemManage**, **BusFault**, **UsageFault**, and **SVCall**), the function writes to the appropriate system handler priority register (1-3) and returns **OK**; otherwise, it returns **ERROR**.

excVecPrioGet( )

This function returns the priority of the given system exception, excluding the PendSV and SysTick interrupts, by reading the appropriate system handler priority register (1-3) (for **MemManage**, **BusFault**, **UsageFault**, and **SVCall**) or retrieving the fixed priority (for reset, NMI, and HardFault) and writing it to the given address. The function returns **OK** if a valid system exception is given; otherwise, it returns **ERROR**.

Parent topic: Architecture Considerations on page 26

2.6. Non-Zero-Based Exception Vectors

BSPs with exception vectors that are not based at address zero can specify the location of the exception vectors by calling the excVecBaseSet( ) function from usrInit( ) with an address argument.

The address supplied as an argument is the base address of the exception vectors. If this function is not called, the exception vectors default to the base address of 0x00000000.

Parent topic: Architecture Considerations on page 26

2.7. Exception Pointer Table Addressing

Exception pointer table addressing is not supported in VxWorks for ARMv7-M.

Parent topic: Architecture Considerations on page 26

2.8. Divide-by-Zero Handling

This exception is enabled (using the configuration and control register) during system initialization to generate a use fault.

WNDRVR

The ARMv7-M architecture allows trapping a divide-by-zero result.

Parent topic: Architecture Considerations on page 26

2.9. Software Floating-Point Support

For the Cortex M3 core, hardware floating-point is unsupported.

To perform floating-point arithmetic, VxWorks for Cortex-M3 instead relies on highly tuned software modules. These modules are automatically linked into the VxWorks kernel and are available to any application that requires floating-point support..

Parent topic: Architecture Considerations on page 26

2.10. Vector Floating-Point Support

For the Cortex-M4F core, vector floating-point is supported.

About This Task

This is a single-precision implementation of the VFPv4-D16. The floating-point extension is called the FPv4-SP extension. Based on the VFP implementation options defined for the ARMv7-A and ARMv7-R architecture profiles, a full characterization of the ARMv7-M floating-point extension is FPv4-SP-D16-M.

Vector floating-point support is enabled as follows:

Procedure

1. When creating your VSB, select the FP_ARMARCH7M_vector floating-point configuration option.
2. Add VX_VFP_TASK to the options field of taskInit( ) or taskSpawn( ).

Parent topic: Architecture Considerations on page 26

2.11. Memory Layout

The VxWorks memory layout for ARMv7-M (real or virtual, as appropriate) shares the same labels as that of the ARM memory layout.

However, the memory layout for ARMv7-M includes some differences. For some ARMv7-M boards, the loadable VxWorks image is not supported. This means that the VxWorks ROM resident image is used. The figure below shows memory layout, labeled as follows:

System Memory Pool

Size depends on size of the system image. The sysMemTop( ) function returns the end of the free memory pool.

System Image

VxWorks itself (three sections: text, data, and bss). The entry point for VxWorks is at the start of this region.

Initial Stack

Initial stack for usrInit( ), until usrRoot( ) is allocated a stack.

Exception Message

ASCII string of fatal exception message.

Boot Line

ASCII string of boot parameters.

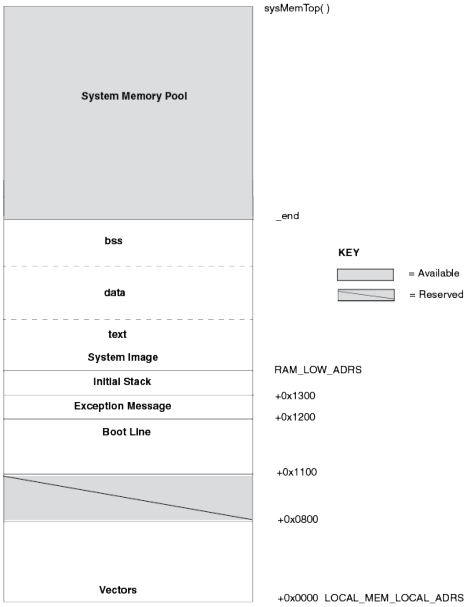
Vectors

Table of exception/interrupt vectors.

> **Note:** The default location of the vectors section is specified by calling the excVecBaseSet( ) function. For more information, see Non-Zero-Based Exception Vectors on page 28.

All addresses shown are relative to the start of memory for a particular target board. The start of memory (corresponding to 0x0 in the memory layout diagram) is defined as **LOCAL_MEM_LOCAL_ADRS** under **INCLUDE_MEMORY_CONFIG** for each target.

> **Note:** The initial stack and system image addresses are configured within the BSP.



Parent topic: Architecture Considerations on page 26

2.12. Expose the Target Memory Map Programmatically

The BSP statically defines the memory regions that are available to VxWorks while it is running on this target. For example, on the **ti_sitara_a8** BSP, the memory map for the **am335x-evmsk** board is defined in the file **am335x-evmsk.dts**.

```
memory
    {
    device_type = "memory";
    reg = <0x80000000 0x10000000>; /* on board 256MB */
    };
```

The memory map is also available programmatically to the VxWorks application, BSP and driver code. The VxWorks API function sysMemDescGet( ) must be called to retrieve the memory map configuration while the target is running.

For example:

```
    LOCAL PHYS_MEM_DESC sysPhysRamDesc[32];
    LOCAL STATUS success;
```

```
success = sysMemDescGet(MEM_DESC_RAM, 0, sysPhysRamDesc);
```

Parent topic: Architecture Considerations on page 7

Parent topic: Architecture Considerations on page 26

3. Reference Material

Comprehensive information regarding ARMv7-M hardware behavior and programming is available from ARM and other sources.

ARM Ltd. provides several hardware and programming manuals for the ARM processor on its website:

http://infocenter.arm.com/help/index.jsp

Wind River recommends that you consult the hardware documentation for your processor or processor family as necessary during BSP development.

ARM Development Reference Documents

The information given in this section is current at the time of writing; should you decide to use these documents, you may want to contact the manufacturer for the most current version.

- *ARM Architecture Reference Manual*, ARM DDI 0100 I.

  Note: This document describes the architecture in general, including architectural standards for instruction bit fields. More specific information is found in the data sheets for individual processors, which conform to different architecture specification versions.

- *ARM Architecture Reference Manual*, ARMv7-A and ARMv7-R edition, ARM DDI 0406 B.
- *ARM System Architecture*, by Steve Furber. Addison-Wesley, 1996. ISBN 0-201-403352-8.
- *ARM Procedure Call Standard (APCS)*, a version of which is available on the Internet. Contact ARM for information on the latest version.
- *ARMv7-M Architecture Reference Manual* (ARM DDI 0403)
- *Cortex™-M3 Technical Reference Manual* (r2p0)
- *Cortex™-M4 Technical Reference Manual* (r0p1)

4. ARM64

1. Interface Variation

1.1. Interface Variations

Particular features and functions that are specific to ARM64 targets.

This section describes the features and functions that are specific to the ARM64 targets, insofar as they:

- are available only for ARM64 targets
- use parameters specific to ARM64 targets
- have special restrictions or characteristics on ARM64 targets

For more complete documentation on the APIs, see the individual reference entries.

1.2. Supported Processors

VxWorks for ARM64 supports ARM Architecture Version 8-A CPUs running in exception level EL1, in little-endian mode.

The ARM Architecture Version 8-A (little-endian, AARCH64) Cortex-A53/A57/A72 processor core are supported with this release.

Note: Clang (LLVM) is compiler for ARM64. For more information, see Compiler Considerations on page 43 and http://clang.llvm.org/.

1.3. ILP32 and LP64 Data Models

Compare 32 and 64-bit models for various data types.

The traditional data model for 32-bit processors, and supported on VxWorks, is the ILP32 data model (where I means integer, L means long, and P means pointer). The value 32 refers to the 32-bit size of those three data types.

The 64-bit data model selected for the VxWorks 64-bit platform is the LP64 data model. The value 64 refers to the 64-bit size of the long and pointer data types.

The following table illustrates the differences between the two data models.

Table 1. Data Types and Sizes for IPL32 and LP64

Type	ILP32 Size	LP64 Size
char	8	8
short	16	16
int	32	32
long	32	64
long long	64	64

Type	ILP32 Size	LP64 Size
pointer	32	64
off_t	32/64	64
size_t	32	64
float	32	32
double	64	64
long double	64	128
enum	32	32

Since the size of the char, short, and int data types do not differ between the two data models, porting applications developed for a 32-bit environment is straightforward for these data types.

The pointer type is different. You can no longer assume that an int type variable can hold a pointer or any address.

The long long type comes from C99. In the LP64 data model, it does not differ from the long data type. In the ILP32 data model, however, it does differ.

The off_t and size_t types are POSIX types that represent an offset and a size, respectively. They have been introduced for the data size neutrality of some APIs That is, you do not have to know what size of objects those types represent in order to use them.

For the ILP32 data model, the off_t type has either a 32-bit size or a 64-bit size. This is specific to VxWorks. In the kernel environment the size of off_t is 32 bits, whereas in the RTP environment it is 64 bits.

The long double type is another C99 type. Its size is not defined by the standard other than being at least equal to that of the double type. For the ILP32 data model, the Clang (LLVM) compiler implements it as a 64-bit type on ARM32. For the LP64 data model, the compiler implements it as a 128-bit type on ARM64.

1.4. Restrictions on cret( ) and tt( )

The cret( ) and tt( ) functions make assumptions about the standard prolog for functions.

If functions are written in assembly language, or in another language that generates a different prolog, the cret( ) and tt( ) functions may generate unexpected results.

The tt( ) does not report the parameters to C functions as it cannot identify the parameters from the code generated by the compiler.

1.5. Stack Alignment

The stack alignment for all ARM64 CPUs is 16 bytes.

1.6. dbgLib

ARM supports both software breakpoints and hardware breakpoints.

When you set a software breakpoint, VxWorks replaces an instruction with a known undefined instruction. VxWorks restores the original code when the breakpoint is removed. If memory is examined or disassembled, the original code is displayed. The instruction cache is purged each time VxWorks changes an instruction to a software breakpoint instruction.

A hardware breakpoint uses the processor's debug registers to set the breakpoint. The supported number of hardware breakpoint is defined by the ARM implementation. For example, there are six breakpoints and four watchpoints for Cortex-A53.

If you are using the target shell, you can set hardware breakpoints with the bh( ) function. It takes the following arguments:

breakpointAddress

Specifies the address.

breakpointType

Specifies when to break. Use one of these values:

BRK_INST

Break when an instruction at this address is executed (0x00).

BRK_DATAR

Break when data is read from this address (0x01).

BRK_DATAW

Break when data is written to this address (0x02).

BRK_DATARW

Break when data is read from or written to this address (0x03).

breakpointTask

Specifies the task to set. To specify all tasks, use 0.

count

The number of times to hit the breakpoint before the process breaks.

quiet

Use **FALSE** to print debug information when the process breaks. Use **TRUE** to break without printing debug information.

For more information, see the reference entry for bh( ).

In order to maintain compatibility with hardware-assisted debuggers, the hardware breakpoint can be disabled using the ARM_DBG_HARDWARE_BP VSB configuration option.

1.7. Optimized Libraries

Most VxWorks libraries are compiled from portable C source code, but there are some libraries that are compiled from assembly language for better performance.

Kernel Optimized Libraries

The following libraries are optimized for ARM64 targets in the kernel:

bLib

buffer manipulation library (except the swab( ) function)

ffsLib

find first bit set library

taskALib

task management library (for taskIdSelf( ) only in SMP)

vxAtomicLib

atomic operations library

User Space Optimized Libraries

The following libraries are optimized for ARM64 targets in user space:

bLib

buffer manipulation library (except the swab( ) function)

ffsLib

find first bit set library

vxAtomicLib

atomic operations library

1.8. intALib

intCpuLock( ) and intCpuUnlock( )

The intCpuLock( ) function returns the I bit from the DAIF register as the lock-out key for the interrupt level prior to the call to intCpuLock( ). The intCpuUnlock( ) function takes this value as a parameter. For ARM processors, these functions control the CPU interrupt mask directly. They do not manipulate the interrupt levels in the interrupt controller chip.

intIFLock( ) and intIFUnlock( )

The intIFLock( ) function returns the I and F bits from the DAIF as the lock-out key, in an analogous fashion, and the intIFUnlock( ) function takes that value as a parameter. Like intCpuLock( ) and intCpuUnlock( ), these functions control the CPU interrupt mask directly. The intIFLock( ) function is not a replacement for intCpuLock( ). It should only be used by code (such as FIQ setup code) that requires that both the IRQ and the FIQ be disabled.

1.9. vxALib

vxTas( )

The test-and-set primitive vxTas( ) provides a C-callable interface to the ARM64 **LDXRB**/**STXRB** (load/store exclusive) instructions.

vxPstateGet( )

The vxPstateGet( ) function provides a C-callable interface to read the current processor state. Also see PSTATE on page 38.

2. Architectural Considerations

2.1. Exceptions

The ARMv8-A architecture defines a set of exception levels, from EL0 to EL3. EL0 is the lowest exception level and EL3 is the highest.

A common usage model for the exception levels is:

EL0

Applications

EL1

Kernel and associated functions that are typically described as privileged

EL2

Hypervisor

EL3

Secure monitor

Because EL2 and EL3 are optional, the VxWorks kernel works in EL1. RTPs work in EL0.

The exception vector table is located in text segment. It cannot be configured in a VIP project.

Exceptions are handled as follows: the exception stub calls excExcContinue( ), and excExcContinue( ) calls the installed handler. Handlers can be installed with calls to excVecSet( ). Because all the synchronous exceptions share a same vector for ARM64, the input parameter vector for excVecSet( ) is not the vector address, but it the exception class. All the valid exception classes are defined in

installDir **/vxworks-7/pkgs_v2/os/arch/arm/kernel/armv8a/h/arch/arm/excArmLib.h**.

2.2. Memory Management

In VxWorks 7, the BSP does not handle device memory space. It is handled by the device driver.

The memory information is supplied in the device tree memory node. For more information about DTS, see the VxWorks 7 BSP and Driver Guide. Memory management is generally performed on small pages that are 4 KB in size. Pages of 16 KB and 64 KB in size are not supported.

Page Size Optimization

The optimal MMU page size is determined automatically when memory is mapped. VxWorks for ARM64 only supports a 4 K page size. So 512 continuous 4 K pages with the same memory attribution are optimized to be a 2 MB block. 512 continuous 2M block with the same memory attribution are optimized to be a 1 GB block.

Address Space Identifiers and RTP Limitation

ARM Architecture Version 8 MMUs support both 8 bit and 16 bit unique address space identifiers (ASIDs), which are used to improve context switch performance with real-time processes (RTPs) by reducing the need to flush and invalidate caches. VxWorks ARM64 only supports 8 bits ASIDs (to be consistent with ARMv7). One ASID is reserved for kernel use and one ASID is reserved for global use. 254 RTPs are supported. If no ASIDs are available, rtpSpawn( ) returns ERROR.

Virtual and Physical Addressing Support

VxWorks ARM64 supports up to 48-bits of virtual address space. The virtual address space can be in the following ranges:

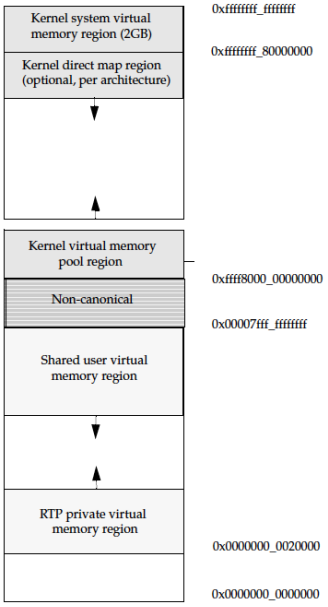
0xFFFF000000000000 ~ 0xFFFFFFFFFFFFFFFF

0x0000000000000000 ~ 0x0000FFFFFFFFFFFF

VxWorks ARM64 supports up to 48-bits of physical address space.

2.3. Memory Layout

The VxWorks memory layout is the same for all ARM64 processors.



Kernel System Virtual Memory Region

The kernel system virtual memory region is located in the top 2 GB of the virtual memory space. That is, from 0xffffffff_80000000 to 0xffffffff_ffffffff. This is where the kernel code segments and the kernel proximity heap, and so on, are located.

Kernel Virtual Memory Pool Region

The kernel virtual memory pool region is the region used for dynamically managed mappings in the kernel. Virtual memory may be carved (for example, for persistent memory) or allocated from this region using the kernel virtual memory pool. This region starts at 0xffff8000_00000000 and grows as needed.

RTP Private Virtual Memory Region

The RTP private virtual memory region is added to each RTP's page manager upon creation. It is used for allocating the RTP's code segments. The remaining portion is also be used for private mappings of the RTP (heap, private mmap( ), and so on). For ARM64 this region starts at a fixed address (0x00000000_00200000) and grows up as needed.

Shared User Virtual Memory Region

The shared user virtual memory region is used for creating the shared virtual memory pool. This region ends at 0x00007fff_ffffffff and grows down as needed.

2.4. Expose the Target Memory Map Programmatically

The BSP statically defines the memory regions that are available to VxWorks while it is running on this target. For example, on the **ti_sitara_a8** BSP, the memory map for the **am335x-evmsk** board is defined in the file **am335x-evmsk.dts**.

```
memory
    {
    device_type = "memory";
    reg = <0x80000000 0x10000000>; /* on board 256MB */
    };
```

The memory map is also available programmatically to the VxWorks application, BSP and driver code. The VxWorks API function sysMemDescGet( ) must be called to retrieve the memory map configuration while the target is running.

For example:

```
    LOCAL PHYS_MEM_DESC sysPhysRamDesc[32];
    LOCAL STATUS success;

    success = sysMemDescGet(MEM_DESC_RAM, 0, sysPhysRamDesc);
```

Parent topic: Architecture Considerations on page 7

Parent topic: Architecture Considerations on page 26

2.5. Miscellaneous Considerations

Execution State

There are two execution states for ARMv8-A architecture, AArch64 and AArch32.

AArch64

The 64-bit execution state. VxWorks ARM64 works in AArch64.

AArch32

The 32-bit execution state. Operation in this state is compatible with ARMv7-A operation. The AArch32 execution state is treated as ARMv7-A in VxWorks.

Stack Management

The task stack is used for task-level execution. The intEnt( ) and intExit( ) functions are used to switch to and from the interrupt stack, if the interrupt stack is enabled. The size of the interrupt stack is determined by the **ISR_STACK_SIZE** macro.

In VxWorks ARM64, the SP_EL1 is used when an exception and interrupt occur. The SP_EL1 points to a small stack area (safeStackArea) that the kernel can guarantee to always be valid. The small safe stack can be used to temporarily save registers, as needed. The SP_EL0 points to a kernel task stack or interrupt stack which is larger, but not guaranteed to be safe from overflow.

Byte Order

The ARM64 byte order is little-endian, but network applications must convert some data to a standard network order, which is big-endian. In particular, in network applications, be sure to convert the port number to network byte order using htons( ).

Registers

PSTATE

PSTATE is not an architectural register for ARMv8. For VxWorks, however, the underlying state as the architectural format used for SPSR. The PSTATE register combines the valid bits of the NZCV, DAIF, CurrentEL and SPSel registers.

X18

The general purpose register X18 is reserved for a VxWorks platform register. User code must not use the X18 register. Kernel and user applications must be compiled with the -ffixed-x18 compiler option (which is one of the defaults for VxWorks builds).

Multi-Cluster

The ARM architecture abstracts the system as a series of inner and outer shareability domains. A cluster is a set of CPUs that share the same inner shareable attribute. Different clusters share the same outer sharable attribute. The multi-cluster feature can reduce complexity and improve the performance for a multi CPUs system. VxWorks ARM64 supports multi-cluster. If a board supports multi-cluster, the feature is enabled by adding the VSB option SUPPORT_MULTI_CLUSTERS to the file **common.vxconfig**. The reg field for the cpus node in the **.dts** file must use this format:

```
cpus {
...
cpu@0 {
            device_type = "cpu";
            compatible = "arm,cortex-a53";
            reg = <0 0>;    /* The first 0 is the cluster number. The second
0                                is the core number in the cluster */
        };

        cpu@1 {
            device_type = "cpu";
            compatible = "arm,cortex-a53";
            reg = <0 1>;
        };
        cpu@2 {
            device_type = "cpu";
            compatible = "arm,cortex-a53";
            reg = <1 0>;
        cpu@3 {
            device_type = "cpu";
            compatible = "arm,cortex-a53";
            reg = <1 1>;
        };
...
};
```

Unaligned Accesses

ARM Architecture Version 8 CPUs support unaligned data access. VxWorks does not enable strict alignment access. For more information, see the *ARM Architecture Reference Manual*.

Divide-by-Zero Handling

There is no native integer divide-by-zero exception on ARM architectures.

Floating-Point Support

VxWorks ARM64 only support hard floating-point. Soft floating-point is not supported. To enable floating-point support use the VX_VFP_TASK option with taskInit( ) or taskSpawn( ).

VFP exceptions are not detected or handled.

The math library used by VxWorks for ARM is licensed by ARM Ltd. For information about the math library, see http://www.arm.com/.

5. MIGRATION FROM ARM32 TO ARM64

1. Porting ARM Assembly Code From 32-Bit to 64-Bit

Assembly code written for ARM32 must be rewritten for ARM64 (ARMv8 AArch64).

While the A64 instruction set has a fixed 32-bit instruction length, and the syntax is similar, there are significant differences for:

- register set structure
- conditional execution of instructions
- multiple memory access instructions
- dedicated function/exception return instruction
- coprocessor support
- CPSR and PSTATE
- ABIs

Note: For information see *Porting C code from 32-Bit to 64-Bit* application note.

Register Set on page 41 The register set for ARM64 provides 31 64-bit general purpose registers. All can be used either as 64-bit Xn registers or 32-bit Wn registers, operating on just the lower 32 bits.
Other Considerations on page 42 In addition to register set structure, there are several other considerations when porting ARM assembly code from 32-bit to 64-bit.

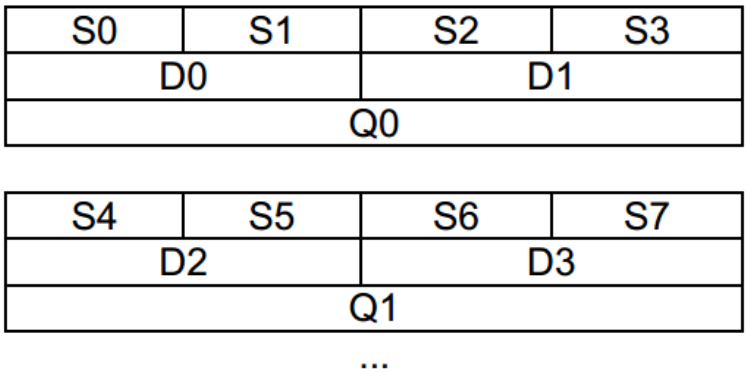
1.1. Register Set

The register set for ARM64 provides 31 64-bit general purpose registers. All can be used either as 64-bit Xn registers or 32-bit Wn registers, operating on just the lower 32 bits.

There is a separate register bank for SIMD and floating point. This offers 32 registers, all 128 bits wide. These can be addressed as vectors of elements ranging from bytes up to a 128-bit quad word.

A key difference for programmers who are used to earlier ARM architectures is the change in the way the registers map. Previously, in the NEON architecture (for example), the mapping of word (S), double word (D) and quad word (Q) registers looked like this:

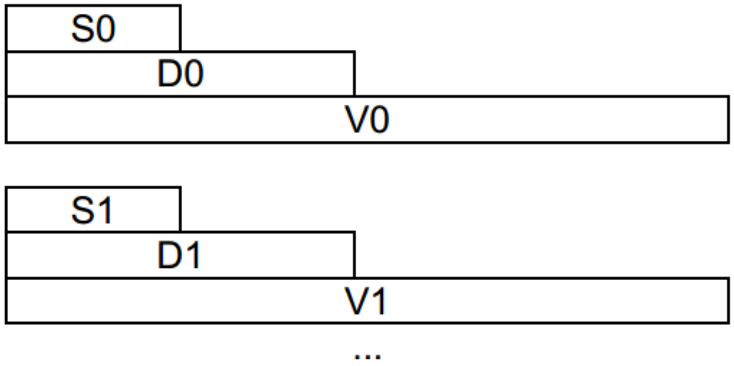
Figure 4. NEON Word Mapping Registers



S0 and S1 combine to form D0; D0 and D1 combine to form Q0; and so on. For some purposes, this is not ideal.

For ARM64, the mapping has changed to this:

Figure 5. A64 Word Mapping Registers



Parent topic: Porting ARM Assembly Code From 32-Bit to 64-Bit on page 41

1.2. Other Considerations

In addition to register set structure, there are several other considerations when porting ARM assembly code from 32-bit to 64-bit.

Conditionally Executed Instructions

Most instructions in the A32 instruction set can be executed conditionally. Only a small number of instructions in the A64 instruction set can be executed conditionally.

Multiple Memory Access Instructions

A64 does not provide multiple memory access instructions. The multiple memory access instructions must be replaced with STP/LDP, or STR/LDR if the number of the registers is odd.

Dedicated Function/Exception Return Instruction

A64 provides the dedicated function/exception return instruction RET/ERET. The RET instruction indicates to the processor that the branch is a function return, which can improve branch prediction (using a return stack) considerably. The program counter (PC) is no longer generally accessible. In particular, it cannot be read or modified like other general purpose registers.

Coprocessors

ARM64 does not support coprocessors.

CPSR

For ARM64, the CPSR is called PSTATE. However, the PSTATE is not an architectural register. It includes the following special registers: NZCV, DAIF, CurrentEL and SPSel.

Different ABIs

For A64 assembly code, the ARM Architecture Procedure Call Standard (AAPCS) must be followed. There are two key points to consider:

WNDRVR

- Register usage. The argument registers for ARM64 are from r0 to r7. The r18 register is reserved for VxWorks as a platform register. The r18 register must not be used in application code.

  Stack alignment. The stack must be 16 bytes aligned for an interface.

Parent topic: Porting ARM Assembly Code From 32-Bit to 64-Bit on page 41

2. Compiler Considerations

Clang (LLVM) is the only supported compiler for ARM64.

Clang (LLVM) is fully compatible with the GNU compiler.

You can use the built-in macro **aarch64** to conditionally compile your code for ARM64 only.

The -Wconversion compiler option is useful when porting C code from 32-bit to 64-bit. It helps to catch implicit casts of incompatible types.

For the LLVM documentation, see http://releases.llvm.org/8.0.0/tools/clang/docs/.

3. Additional Documentation

Several other sources of information about migrating from ARM32 to ARM64 are available.

- *VxWorks 7 Platform Programmer's Guide*
- *ARM® Architecture Reference Manual ARMv8, for ARMv8-A architecture profile*
- *ARM® Architecture Reference Manual ARMv7-A and ARMv7-R edition*
- *Procedure Call Standard for the ARM 64-bit Architecture*
- *Procedure Call Standard for the ARM Architecture*
- *Porting to 64-bit ARM* (https://community.arm.com)